Chapter 11 ▪ Annotations

Annotations are tags that you insert into your source code so that some tool can process them. The tools can operate on the source level, or they can process class files into which the compiler has placed annotations.

Annotations do not change the way your programs are compiled. The Java compiler generates the same virtual machine instructions with or without the annotations.

To benefit from annotations, you need to select a processing tool and use annotations that your processing tool understands. You need to invoke the tool in your build or execution process.

There is a wide range of uses for annotations. For example, JUnit (available at **https://junit.org**) uses annotations to mark methods that execute tests and to specify how the tests should be run. Jakarta Persistence (**https://jakarta.ee/specifications/persistence/**) uses annotations to define mappings between classes and database tables, so that objects can be persisted automatically without the developer having to write SQL queries.

In this chapter, you will learn the details of the annotation syntax, how to define your own annotations, and how to write annotation processors that work at the source level or at runtime.



Note: For a compelling use of annotations, check out `JCommander` (**https://jcommander.org**) and picocli (**https://picocli.info**). These libraries use annotations for the processing of command-line arguments.

The key points of this chapter are:

1. You can annotate declarations just as you use modifiers such as `public` or `static`.
2. You can also annotate types that appear in declarations, casts, `instanceof` checks, or method references.
3. An annotation starts with a `@` symbol and may contain key/value pairs called elements.

4. Annotation values must be compile-time constants: primitive types, `enum` constants, `Class` literals, other annotations, or arrays thereof.
5. A declaration or type can have repeating annotations or annotations of different types.
6. To define an annotation, specify an annotation interface whose methods correspond to the annotation elements.
7. The Java library defines over a dozen annotations, and annotations are extensively used in the Java Enterprise Edition.
8. To process annotations in a running Java program, you can use reflection and query the reflected items for annotations.
9. Annotation processors process source files during compilation, using the Java language model API to locate annotated items.

11.1. Using Annotations

Here is an example of a simple annotation:

```
class CacheTest {
   ...
   @Test public void checkRandomInsertions()
}
```

The annotation `@Test` annotates the `checkRandomInsertions` method. In Java, an annotation is used like a modifier (such as `public` or `static`). The name of each annotation is preceded by an `@` symbol.

By itself, the `@Test` annotation does not do anything. It needs a tool to be useful. When running JUnit, it finds the methods that are annotated with `@Test` and calls them.

11.1.1. Annotation Elements

Annotations can have key/value pairs called *elements*, such as

```
@RepeatedTest(value=10, failureThreshold=3)
```

The names and types of the permissible elements are defined by each annotation (see Section 11.2). The elements can be processed by the tools that read the annotations.

The value of an annotation element is one of the following:

- A primitive type value
- A `String`
- A `Class` object
- An instance of an `enum`
- An annotation
- An array of the preceding (but not an array of arrays)

To illustrate these possibilities, here is a `@BugReport` annotation, not from an actual library:

```
@BugReport(showStopper=true,
   assignedTo="Harry",
   testCase=CacheTest.class,
   status=BugReport.Status.CONFIRMED,
   ref=@Reference(id=11235811),
   reportedBy={"Harry", "Fred"})
```



Note: Since annotations are processed by the compiler, all element values must be compile-time constants.



Caution: An annotation element can never have the value `null`.

If an element value is an array with a single element, you can omit the braces:

```
@BugReport(reportedBy="Harry") // Same as reportedBy={"Harry"}
```

Elements can have default values. For example, the `failureThreshold` element of the JUnit `@RepeatedTest` annotation has default `Integer.MAX_VALUE`. Therefore, the annotation `@RepeatedTest(value=10)` is equivalent to `@RepeatedTest(value=10, failureThreshold=Integer.MAX_VALUE)`. When running this test, it is repeated ten times, no matter how often it failed.

If the element name is `value`, and that is the only element you specify, you can omit `value=`. For example, `@RepeatedTest(10)` is the same as `@RepeatedTest(value=10)`.

11.1.2. Multiple and Repeated Annotations

An item can have multiple annotations:

```
@Test
@Tag("localized")
void testHello()
```

If the author of an annotation declared it to be repeatable, you can repeat the same annotation multiple times:

```
@Tag("localized")
@Tag("showstopper")
void testHello()
```

11.1.3. Annotating Declarations

So far, you have seen annotations applied to method declarations. There are many other places where annotations can occur. They fall into two categories: *declarations* and *type uses*. Declaration annotations can appear at the declarations of

- Classes (including `enum`) and interfaces (including annotation interfaces)
- Methods
- Constructors
- Instance variables (including `enum` constants)
- Local variables (including those declared in `for` and try-with-resources statements)
- Parameter variables and `catch` clause parameters
- Type parameters
- Packages and modules

For classes, interfaces, and modules, put the annotations before the `class`, `interface`, or `module` keyword and any modifiers:

```
@Entity public class User { ... }
```

For variables, put them before the type:

```
@SuppressWarnings("unchecked") List<User> users = ...;
public User getUser(@Param("id") String userId)
```

This also holds for record components:

```
public record Rectangle(@ToString(includeName=false) Point topLeft,
  int width, int height) {}
```

A type parameter in a generic class or method can be annotated like this:

```
public class Cache<@NonNull V> { ... }
```

A package is annotated in a file `package-info.java` that contains only the package statement preceded by annotations.

```
/**
   Package-level documentation
*/
@Generated("com.horstmann.generator")
package com.horstmann.corejava.generated;
import javax.annotation.processing.Generated;
```

Note that the `import` statement for the annotation comes *after* the `package` declaration.



Note: Annotations for local variables and packages are discarded when a class is compiled. Therefore, they can only be processed at the source level.

11.1.4. Annotating Type Uses

A declaration annotation provides some information about the item being declared. For example, in the declaration

```
public User getUser(@NonNull String userId)
```

it is asserted that the `userId` parameter is not `null`.



Note: The `@NonNull` annotation is a part of the Checker Framework (**http://types.cs.washington.edu/checker-framework**). With that framework, you can include assertions in your program, such that a parameter is non-`null` or

that a `String` has been localized—that is, adapted to the user's local language and usage. A static analysis tool then checks whether the assertions are valid in a given body of source code.

Now suppose we have a parameter of type `List<String>`, and we want to express that all of the strings are non-`null`. That is where type use annotations come in. Place the annotation before the type argument: `List<@NonNull String>`

Type use annotations can appear in the following places:

1. With generic type arguments: `List<@NonNull String>`, `Comparator.<@NonNull String> reverseOrder()`.
2. Before array brackets:
   1. `String[] @NonNull [] words`: Applies to `String[]`—the arrays `words[i]` are not `null`
   2. `String @NonNull [][] words`: Applies to `String[][]`—the variable `words` is not `null`

   But note that `@NonNull String[][] words` applies to `String`—the elements `words[i][j]` are not `null`
3. With superclasses and implemented interfaces: `class Warning extends @Localized Message`.
4. With constructor invocations: `new @Localized String(...)`.
5. With nested types: `Map.@Localized Entry`.
6. With casts and `instanceof` checks:

   ```
   (@Localized String) text
   if (text instanceof @Localized String)
   ```

   The annotations are only for use by external tools. They have no effect on the behavior of a cast or an `instanceof` check.
7. With exception specifications: `public String read() throws @Localized IOException`.
8. With wildcards and type bounds: `List<@Localized ? extends Message>`, `List<? extends @Localized Message>`.
9. With method and constructor references: `@Localized Message::getText`.

There are a few type positions that cannot be annotated:

```
@NonNull String.class // Error—cannot annotate class literal
import java.lang.@NonNull String; // Error—cannot annotate import
```

You can place annotations before or after other modifiers such as `private` and `static`. It is customary (but not required) to put type use annotations after other modifiers, and declaration annotations before other modifiers. For example,

```
private @NonNull String text; // Annotates the type use
@Id private String userId; // Annotates the variable
```



Note: As you will see in Section 11.2, an annotation author needs to specify where a particular annotation can appear. If an annotation is permissible both for a variable and a type use, and it is used in a variable declaration, then both the variable and the type use are annotated. For example, consider

```
public User getUser(@NonNull String userId)
```

if `@NonNull` can apply both to parameters and to type uses, the `userId` parameter is annotated, and the parameter type is `@NonNull String`.

11.1.5. Making Receivers Explicit

Suppose you want to annotate parameters that are not being mutated by a method.

```
public class Point {
   public boolean equals(@Const Object other) { ... }
}
```

A tool that processes this annotation would reason that `q` has not been changed in the call

```
p.equals(q)
```

But what about `p`?

When the method is called, the receiver variable `this` is bound to `p`, but `this` is never declared, so you cannot annotate it.

Actually, you can declare it, with a rarely used syntax variant, just so that you can add an annotation:

```
public class Point {
   public boolean equals(@Const Point this, @Const Object other) { ... }
}
```

The first parameter is called the *receiver parameter*. It must be named `this`. Its type is the class that is being constructed.



Note: You can provide a receiver parameter only for methods, not for constructors. Conceptually, the `this` reference in a constructor is not an object of the given type until the constructor has completed. Instead, an annotation placed on the constructor describes a property of the constructed object.

A different hidden parameter is passed to the constructor of an inner class, namely the reference to the enclosing class object. You can make this parameter explicit as well:

```
static class Sequence {
   private int from;
   private int to;

   class Iterator implements java.util.Iterator<Integer> {
      private int current;

      public Iterator(@NonNull Sequence Sequence.this) {
         this.current = Sequence.this.from;
      }
      ...
   }
   ...
}
```

The parameter must be named just like when you refer to it, `EnclosingClass.this`, and its type is the enclosing class.

11.2. Defining Annotations

Each annotation must be declared by an *annotation interface*, with the `@interface` syntax. The methods of the interface correspond to the elements of the annotation. For example, the JUnit RepeatedTest annotation is defined by the following interface:

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RepeatedTest {
   int failureThreshold();
   ...
}
```

The `@interface` declaration creates an actual Java interface. Tools that process annotations receive objects that implement the annotation interface. When the JUnit test runner tool gets an object that implements `Test`, it simply invokes the `failureThreshold` method to retrieve the failure threshold of a particular `RepeatedTest` annotation.

The element declarations in the annotation interface are actually method declarations. The methods of an annotation interface can have no parameters and no `throws` clauses, and they cannot be generic.

The `@Target` and `@Retention` annotations are *meta-annotations*. They annotate the `RepeatedTest` annotation, indicating the places where the annotation can occur and where it can be accessed.

The value of the `@Target` meta-annotation is an array of `ElementType` objects, specifying the items to which the annotation can apply. You can specify any number of element types, enclosed in braces. For example,

```
@Target({ElementType.FIELD, ElementType.RECORD_COMPONENT, ElementType.TYPE})
public @interface ToString
```

Table 11.1 shows all possible targets. The compiler checks that you use an annotation only where permitted. For example, if you apply `@ToString` to a method, a compile-time error results.



Note: An annotation without an `@Target` restriction can be used with any declarations but not with type parameters and type uses. (These were the only possible targets in the first Java release that supported annotations.)

Table 11.1: Element Types for the `@Target` Annotation

Element type	Annotation applies to
`ANNOTATION_TYPE`	Annotation type declarations
`PACKAGE`	Packages
`MODULE`	Modules
`TYPE`	Classes (including `enum`) and interfaces (including annotation types)
`METHOD`	Methods
`CONSTRUCTOR`	Constructors
`FIELD`	Instance variables (including `enum` constants)
`PARAMETER`	Method or constructor parameters
`RECORD_COMPONENT`	Record components
`LOCAL_VARIABLE`	Local variables
`TYPE_PARAMETER`	Type parameters
`TYPE_USE`	Uses of a type

When you annotate a record component, the annotation is applied to the fields and methods to which the component gives rise, provided the annotation is permitted on fields or methods. For example, in the annotation

```
public record Rectangle(@ToString(includeName=false) Point topLeft,
  int width, int height) {}
```

the `topLeft` field receives the annotation if its target types include `ElementType.FIELD`.

The `@Retention` meta-annotation specifies where the annotation can be accessed. There are three choices.

1. `RetentionPolicy.SOURCE`: The annotation is available to source processors, but it is not included in class files.
2. `RetentionPolicy.CLASS`: The annotation is included in class files, but the virtual machine does not load them. This is the default.
3. `RetentionPolicy.RUNTIME`: The annotation is available at runtime and can be accessed through the reflection API.

You will see examples of all three scenarios later in this chapter.

There are several other meta-annotations—see **Section 11.3** for a complete list.

To specify a default value for an element, add a `default` clause after the method defining the element. For example,

```
public @interface RepeatedTest {
   int failureThreshold() default Integer.MAX_VALUE;
   ...
}
```

This example shows how to denote a default of an empty array and a default for an annotation:

```
public @interface BugReport {
   String[] reportedBy() default {};
      // Defaults to empty array
   Reference ref() default @Reference(id=0);
      // Default for an annotation
   ...
}
```



Caution: Defaults are not stored with the annotation; instead, they are dynamically computed. If you change a default and recompile the annotation class, all annotated elements will use the new default, even in class files that have been compiled before the default changed.

All annotation interfaces implicitly extend the `java.lang.annotation.Annotation` interface. That interface is a regular interface, *not* an annotation interface.

You cannot extend annotation interfaces, and you never supply classes that implement annotation interfaces. Instead, source processing tools and the virtual machine generate proxy classes and objects when needed.

11.3. Annotations in the Java API

The Java API defines a number of annotation interfaces in the `java.lang`, `java.lang.annotation`, and `javax.annotation` packages. Four of them are meta-annotations that describe the behavior of annotation interfaces. The others are regular annotations that you use to annotate items in your source code. **Table 11.2** shows these annotations. I will discuss them in detail in the following two sections.

Table 11.2: Annotations in the Java API

Annotation interface	Applicable to	Purpose
`Override`	Methods	Checks that this method overrides a superclass method.
Serial	Methods	Checks that this method is a correct serialization method.
`Deprecated`	All declarations	Marks item as deprecated.
SuppressWarnings	All declarations except packages	Suppresses warnings of a given type.
`SafeVarargs`	Methods and constructors	Asserts that the varargs parameter is safe to use.

Annotation interface	Applicable to	Purpose
FunctionalInterface	Interfaces	Marks an interface as functional (with a single abstract method).
`Generated`	All declarations	Marks an item as source code that has been generated by a tool.
Target	Annotations	Specifies the locations to which this annotation can be applied.
`Retention`	Annotations	Specifies where this annotation can be used.
Documented	Annotations	Specifies that this annotation should be included in the documentation of annotated items.
`Inherited`	Annotations	Specifies that this annotation is inherited by subclasses.
Repeatable	Annotations	Specifies that this annotation can be applied multiple times to the same item.

11.3.1. Annotations for Compilation

The `@Deprecated` annotation can be attached to any items whose use is no longer encouraged. The compiler will warn when you use a deprecated item. This annotation has the same role as the `@deprecated` JavaDoc tag. However, the annotation persists until runtime.



Note: The `jdeprscan` utility which is part of the JDK can scan a set of JAR files for deprecated elements.

The `@Override` annotation makes the compiler check that the annotated method really overrides a method from the superclass. For example, if you declare

```
public class Point {
   @Override public boolean equals(Point other) { ... }
   ...
}
```

then the compiler will report an error—this `equals` method does not override the `equals` method of the `Object` class because that method has a parameter of type `Object`, not `Point`.

The `@Serial` annotation checks that methods used for serialization, which are not declared in interfaces, have the correct parameter types.

The `@SuppressWarnings` annotation tells the compiler to suppress warning messages of a particular category, for example

```
@SuppressWarnings("unchecked") T[] result
   = (T[]) Array.newInstance(cl, n);
```

The `@SafeVarargs` annotation asserts that a method does not corrupt its varargs parameter (see **Chapter 6**).

The `@Generated` annotation is intended for use by code generator tools. Any generated source code can be annotated to differentiate it from programmer-provided code. For example, a code editor can hide the generated code, or a code generator can remove older versions of generated code. Each annotation must contain a unique identifier for the code generator. A date string (in ISO 8601/RFC 3339 format) and a comment string are optional. For example,

```
@Generated(value="com.horstmann.generator",
   date="2015-01-04T12:08:56.235-0700");
```

You have seen the `FunctionalInterface` annotation in **Chapter 3**. It is used to annotate conversion targets for lambda expressions, such as

```
@FunctionalInterface
public interface IntFunction<R> {
   R apply(int value);
```

```
}
```

If you later add another abstract method, the compiler will generate an error.

Of course, you should only add this annotation to interfaces that describe functions. There are other interfaces with a single abstract method (such as `AutoCloseable`) that are not conceptually functions.

11.3.2. Meta-Annotations

You have already seen the `@Target` and `@Retention` meta-annotations in **Section 11.2**.

The `@Documented` meta-annotation gives a hint to documentation tools such as JavaDoc. Documented annotations should be treated just like other modifiers (such as `private` or `static`) for documentation purposes. In contrast, other annotations should not be included in the documentation.

For example, the `@SuppressWarnings` annotation is not documented. If a method or field has that annotation, it is an implementation detail that is of no interest to the JavaDoc reader. On the other hand, the `@FunctionalInterface` annotation is documented since it is useful for the programmer to know that the interface is intended to describe a function. **Figure 11.1** shows the documentation.

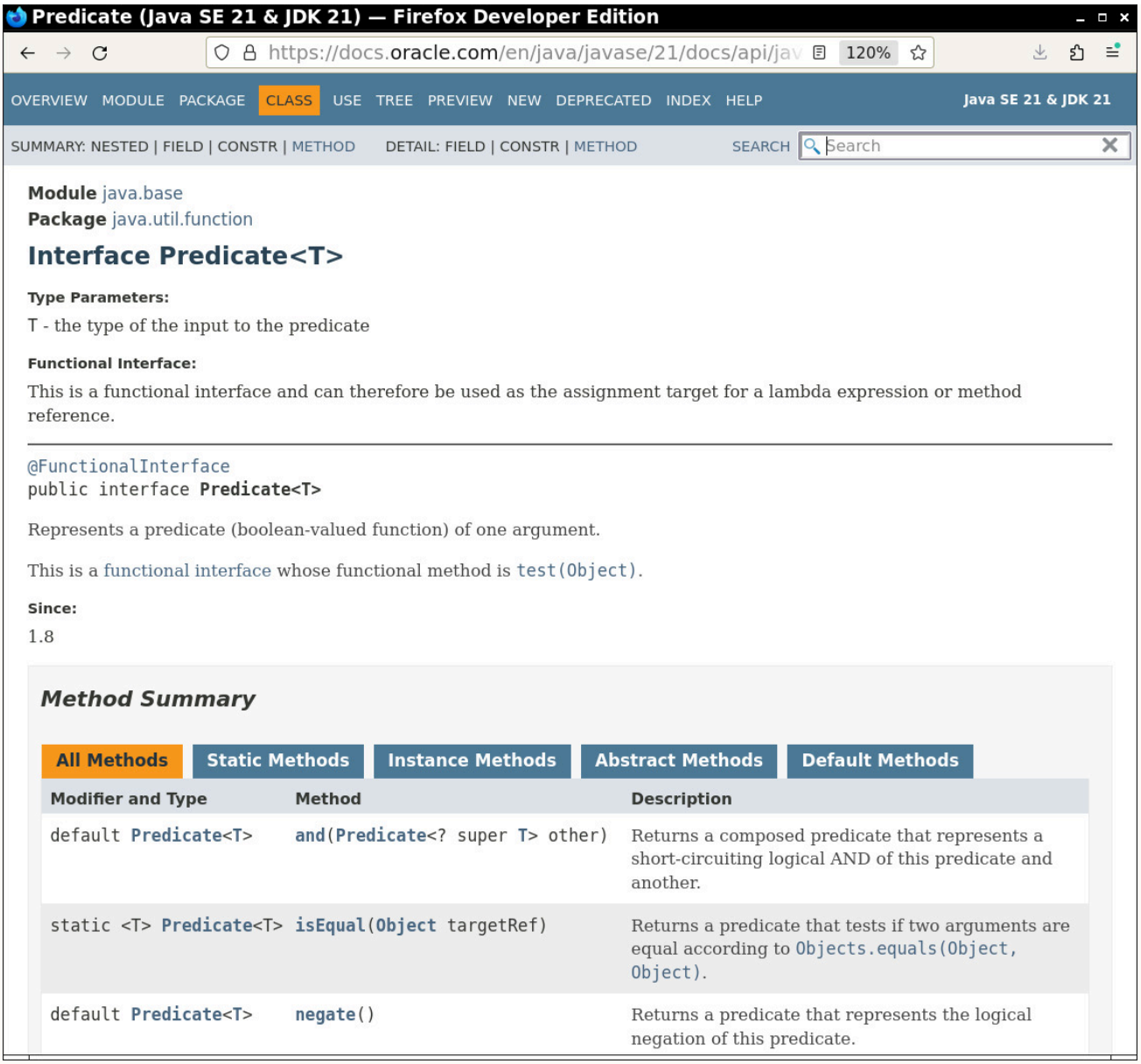


Figure 11.1: A documented annotation

The `@Inherited` meta-annotation applies only to annotations for classes. When a class has an inherited annotation, then all of its subclasses automatically have the same annotation. This makes it easy to create annotations that work similar to marker interfaces (such as the `Serializable` interface).

Suppose you define an inherited annotation `@Persistent` to indicate that objects of a class can be saved in a database. Then the subclasses of persistent classes are automatically annotated as persistent.

```
@Inherited @interface Persistent { }

@Persistent class Employee { ... }
class Manager extends Employee { ... } // Also @Persistent
```

The `@Repeatable` meta-annotation makes it possible to apply the same annotation multiple times. For example, the JUnit `@Tag` annotation is repeatable. It can be used like this:

```
@Tag("localized")
@Tag("showstopper")
void testHello() { ... }
```

For historical reasons, the implementor of a repeatable annotation needs to provide a *container annotation* that holds the repeated annotations in an array.

Here is how to define the `@Tag` annotation and its container:

```
@Repeatable(Tag.class)
@interface Tag
{
  String value();
}

@interface Tags
{
  Tag[] value();
}
```

Whenever the user supplies two or more `@Tag` annotations, they are automatically wrapped into a `@Tags` annotation. This complicates processing of the annotation, as you will see in the next section.

11.4. Processing Annotations at Runtime

So far, you have seen how to add annotations to source files and how to define annotation types. Now the time has come to see what good can come out of that.

In this section, I show you a simple example of processing an annotation at runtime using the reflection API that you have already seen in **Chapter 4**. Suppose we want to reduce the tedium of implementing `toString` methods. Of course, one can write a generic `toString` method using reflection that simply includes all instance variable names and values. But suppose we want to customize that process. We may not want to include all instance variables, or we may want to skip class and variable names. For example, for the

`Point` class we may prefer `[5,10]` instead of `Point[x=5,y=10]`. Of course, any number of other enhancements would be plausible, but let's keep it simple. The point is to demonstrate what an annotation processor can do.

Annotate all classes that you want to benefit from this service with the `@ToString` annotation (which you can find in the `com.horstmann.annotations.runtime` package in the book's companion code). In addition, all instance variables that should be included need to be annotated as well. The annotation is defined like this:

```
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
public @interface ToString {
   boolean includeName() default true;
}
```

Here are annotated `Point` and `Rectangle` classes:

```
@ToString(includeName=false)
public class Point {
   @ToString(includeName=false) private int x;
   @ToString(includeName=false) private int y;
   ...
}

@ToString
public record Rectangle(
  @ToString(includeName=false) Point topLeft,
  @ToString int width,
  @ToString int height) {
}
```

The intent is for a rectangle to be represented as string as `Rectangle[[5, 10],width=20,height=30]`.

At runtime, we cannot modify the implementation of the `toString` method for a given class. Instead, let us provide a method that can format any object, discovering and using the `ToString` annotations if they are present.

The key are the methods

```
T getAnnotation(Class<T>)
T getDeclaredAnnotation(Class<T>)
T[] getAnnotationsByType(Class<T>)
T[] getDeclaredAnnotationsByType(Class<T>)
Annotation[] getAnnotations()
Annotation[] getDeclaredAnnotations()
boolean isAnnotationPresent(Class<? extends Annotation> annotationClass)
```

of the `AnnotatedElement` interface. The reflection classes `Class`, `Field`, `Parameter`, `Method`, `Constructor`, and `Package` implement that interface.

As with other reflection methods, the methods with `Declared` in their name yield annotations in the class itself, whereas the others include inherited ones. In the context of annotations, this means that the annotation is `@Inherited` and applied to a superclass.

If an annotation is not repeatable, call `getAnnotation` to locate it. For example:

```
Class<?> cl = obj.getClass();
ToString ts = cl.getAnnotation(ToString.class);
if (ts != null && ts.includeName()) ...
```

Note that you pass the class object for the annotation (here, `ToString.class`) and you get back an object of some proxy class that implements the `ToString` interface. You can then invoke the interface methods to get the values of the annotation elements. If the annotation is not present, the `getAnnotation` method returns `null`.

It gets a bit messy if an annotation is repeatable. If you call `getAnnotation` to look up a repeatable annotation, and the annotation was actually repeated, then you also get `null`. That is because the repeated annotations were wrapped inside the container annotation.

In this case, you should call `getAnnotationsByType`. That call "looks through" the container and gives you an array of the repeated annotations. If there was just one annotation, you get it in an array of length 1. With this method, you don't have to worry about the container annotation.

The `getAnnotations` method gets all annotations (of any type) with which an item is annotated, with repeated annotations wrapped into containers.

Here is the implementation of the annotation-aware `toString` method:

```
public class ToStrings {
   public static String toString(Object obj) {
      if (obj == null) return "null";
      Class<?> cl = obj.getClass();
      ToString ts = cl.getAnnotation(ToString.class);
      if (ts == null) return obj.toString();
      var result = new StringBuilder();
      if (ts.includeName()) result.append(cl.getSimpleName());
      result.append("[");
      boolean first = true;
      for (Field f : cl.getDeclaredFields()) {
         ts = f.getAnnotation(ToString.class);
         if (ts != null) {
            if (first) first = false; else result.append(",");
            f.setAccessible(true);

            if (ts.includeName()) {
               result.append(f.getName());
               result.append("=");
            }
            try {
               result.append(ToStrings.toString(f.get(obj)));
            } catch (ReflectiveOperationException ex) {
               ex.printStackTrace();
            }
         }
      }
      result.append("]");
      return result.toString();
   }
}
```

When a class is annotated with `ToString`, the method iterates over its fields and prints the ones that are also annotated. If the `includeName` element is `true`, then the class or field name is included in the string.

Note that the method calls itself recursively. Whenever an object belongs to a class that isn't annotated, its regular `toString` method is used and the recursion stops.

This is a simple but typical use of the runtime annotation API. Look up classes, fields, and so on, using reflection; call `getAnnotation` or `getAnnotationsByType` on the potentially annotated elements to retrieve the annotations; then, invoke the methods of the annotation interfaces to obtain the element values.

11.5. Source-Level Annotation Processing

In the preceding section, you saw how to analyze annotations in a running program. Another use for annotation is the automatic processing of source files to produce more source code, configuration files, scripts, or whatever else one might want to generate.

To show you the mechanics, I will repeat the example of generating `toString` methods. However, this time, let's generate them in Java source. Then the methods will get compiled with the rest of the program, and they will run at full speed instead of using reflection.

11.5.1. Annotation Processors

Annotation processing is integrated into the Java compiler. During compilation, you can *invoke annotation processors* by running

javac -processor ProcessorClassName$_1$,ProcessorClassName$_2$,... sourceFiles

The compiler locates the annotations of the source files. Each annotation processor is executed in turn and given the annotations in which it expressed an interest. If an annotation processor creates a new source file, the process is repeated. Once a processing round yields no further source files, all source files are compiled.



Note: An annotation processor can only generate new source files. It cannot modify an existing source file.

An annotation processor implements the `Processor` interface, generally by extending the `AbstractProcessor` class. You need to specify which annotations your processor supports. For our next example, we implement a `ToString` annotation in the `com.horstmann.annotations.source` package:

```
@SupportedAnnotationTypes("com.horstmann.annotations.source.ToString")
@SupportedSourceVersion(SourceVersion.RELEASE_8)
public class ToStringAnnotationProcessor extends AbstractProcessor {
   public boolean process(Set<? extends TypeElement> annotations,
         RoundEnvironment currentRound) {
      ...
   }
}
```

A processor can claim specific annotation types, wildcards such as `"com.horstmann.*"` (all annotations in the `com.horstmann` package or any subpackage), or even `"*"` (all annotations).

The `process` method is called once for each round, with the set of all annotations that were found in any files during this round, and a `RoundEnvironment` reference that contains information about the current processing round.

11.5.2. The Language Model API

You use the *language model* API for analyzing source-level annotations. Unlike the reflection API, which presents the virtual machine representation of classes and methods, the language model API lets you analyze a Java program according to the rules of the Java language.

The compiler produces a tree whose nodes are instances of classes that implement the `javax.lang.model.element.Element` interface and its subinterfaces, `TypeElement`, `VariableElement`, `ExecutableElement`, and so on. These are the compile-time analogs to the `Class`, `Field`/`Parameter`, `Method`/`Constructor` reflection classes.

I do not want to cover the API in detail, but here are the highlights that you need to know for processing annotations.

- The `RoundEnvironment` gives you a set of all elements annotated with a particular annotation, by calling one of the methods

```
Set<? extends Element> getElementsAnnotatedWith(Class<? extends Annotation> a)
Set<? extends Element> getElementsAnnotatedWithAny(
   Set<Class<? extends Annotation>> annotations)
```

The second method is useful for repeated annotations.

- The source-level equivalent of the `AnnotatedElement` interface is `AnnotatedConstruct`. You use the methods

```
A getAnnotation(Class<A> annotationType)
A[] getAnnotationsByType(Class<A> annotationType)
```

to get the annotation or repeated annotations for a given annotation class.

- A `TypeElement` represents a class or interface. The `getEnclosedElements` method yields a list of its fields and methods.
- Calling `getSimpleName` on an `Element` or `getQualifiedName` on a `TypeElement` yields a `Name` object that can be converted to a string with `toString`.

11.5.3. Using Annotations to Generate Source Code

Let us return to our task of automatically generating `toString` methods. We can't put these methods into the original classes—annotation processors can only produce new classes, not modify existing ones.

Therefore, we'll add all methods into a utility class `ToStrings` that the processor will place in the `com.horstmann.annotations.processed` class:

```
public class ToStrings {
   public static String toString(Point obj) {
      Generated code
   }
   public static String toString(Rectangle obj) {
      Generated code
   }
   ...
   public static String toString(Object obj) {
      return Objects.toString(obj);
   }
}
```

Since we don't want to use reflection, we annotate accessor methods, not fields:

```
@ToString
public class Rectangle {
   ...
   @ToString(includeName=false) public Point getTopLeft() { return topLeft; }
   @ToString public int getWidth() { return width; }
   @ToString public int getHeight() { return height; }
}
```

The annotation processor should then generate the following source code:

```
public static String toString(Rectangle obj) {
   var result = new StringBuilder();
   result.append("Rectangle");
   result.append("[");
   result.append(toString(obj.getTopLeft()));
   result.append(",");
   result.append("width=");
   result.append(toString(obj.getWidth()));
   result.append(",");
   result.append("height=");
   result.append(toString(obj.getHeight()));
   result.append("]");
   return result.toString();
}
```

The "boilerplate" code is in gray. Here is an outline of the method that produces the `toString` method for a class with given `TypeElement`:

```
private void writeToStringMethod(PrintWriter out, TypeElement te) {
   String className = te.getQualifiedName().toString();
   Print method header and declaration of string builder
   ToString ann = te.getAnnotation(ToString.class);
   if (ann.includeName()) Print code to add class name
   for (Element c : te.getEnclosedElements()) {
      ann = c.getAnnotation(ToString.class);
      if (ann != null) {
         if (ann.includeName()) Print code to add field name
         Print code to append toString(obj.methodName())
      }
```

```
    }
    Print code to return string
  }
```

And here is an outline of the `process` method of the annotation processor. It creates a source file for the helper class and writes the class header and one method for each annotated class.

```
public boolean process(Set<? extends TypeElement> annotations,
      RoundEnvironment currentRound) {
   if (annotations.size() == 0) return true;
   try {
      JavaFileObject sourceFile = processingEnv.getFiler().createSourceFile(
         "com.horstmann.annotations.processed.ToStrings");
      try (var out = new PrintWriter(sourceFile.openWriter())) {
         Print code for package and class
         for (Element e : currentRound.getElementsAnnotatedWith(ToString.class)) {
            if (e instanceof TypeElement te) {
               writeToStringMethod(out, te);
            }
         }
         Print code for toString(Object)
      }
   } catch (IOException ex) {
      processingEnv.getMessager().printMessage(
         Kind.ERROR, ex.getMessage());
   }
   return true;
}
```

For the tedious details, check the book's companion code.

Note that the `process` method is called in subsequent rounds with an empty list of annotations. It then returns immediately so it doesn't create the source file twice.



Tip: To see the rounds, run the `javac` command with the `-XprintRounds` flag:

```
Round 1:
 input files: {ch11.sec05.Point, ch11.sec05.Rectangle,
  ch11.sec05.SourceLevelAnnotationDemo}
 annotations: [com.horstmann.annotations.source.ToString]
```

```
  last round: false
Round 2:
  input files: {com.horstmann.annotations.processed.ToStrings}
  annotations: []
  last round: false
Round 3:
  input files: {}
  annotations: []
  last round: true
```

This example demonstrates how tools can harvest source file annotations to produce other files. The generated files don't have to be source files. Annotation processors may choose to generate XML descriptors, property files, shell scripts, HTML documentation, and so on.



Note: You have now seen how to process annotations in source files and in a running program. A third possibility is to process annotations in class files, usually on the fly when loading them into the virtual machine. You need a tool such as ASM (**http://asm.ow2.org**) to locate and evaluate the annotations, and rewrite the byte codes.

Exercises

1. Describe how `Object.clone` could be modified to use a `@Cloneable` annotation instead of the `Cloneable` marker interface.
2. If annotations had existed in early versions of Java, then the `Serializable` interface would surely have been an annotation. Implement a `@Serializable` annotation. Choose a text or binary format for persistence. Provide classes for streams or readers/writers that persist the state of objects by saving and restoring all fields that are primitive values or themselves serializable. Don't worry about cyclic references for now.
3. Repeat the preceding assignment, but do worry about cyclic references.
4. Add a `@Transient` annotation to your serialization mechanism that acts like the `transient` modifier.
5. Define an annotation `@Todo` that contains a message describing whatever it is that needs to be done. Define an annotation processor that produces a reminder list from a source file. Include a description of the annotated item and the todo message.
6. Turn the annotation of the preceding exercise into a repeating annotation.

7. If annotations had existed in early versions of Java, they might have taken the role of JavaDoc. Define annotations `@Param`, `@Return`, and so on, and produce a basic HTML document from them with an annotation processor.
8. Implement the `@TestCase` annotation, generating a source file whose name is the name of the class in which the annotation occurs, followed by `Test`. For example, if `MyMath.java` contains

   ```
   @TestCase(args="4", expected="24")
   @TestCase(args="0", expected="1")
   public static long factorial(int n) { ... }
   ```

   then generate a file `MyMathTest.java` with statements

   ```
   assert(MyMath.factorial(4) == 24);
   assert(MyMath.factorial(0) == 1);
   ```

   You may assume that the test methods are static, and that `args` contains a comma-separated list of parameters of the correct type.
9. Implement the `@TestCase` annotation as a runtime annotation and provide a tool that checks it. Again, assume that the test methods are static and restrict yourself to a reasonable set of parameter and return types that can be described by strings in the annotation elements.
10. Implement a processor for the `@Resource` annotation that accepts an object of some class and looks for fields of type `String` annotated with `@Resource(name="URL")`. Then load the URL and "inject" the string variable with that content, using reflection.
11. The `@ToString` annotation in **Section 11.4** only writes the simple name of the class. Add an annotation element for writing either the simple or the full class name.
12. The `@ToString` annotation in **Section 11.5.3** places the generated source files into the `com.horstmann.annotations.processed` package. Improve it so that the files are placed in a package specified in the annotation. You may need to generate more than one source file.